

Assignment 03

ElGamal + ECC + Diffie-Hellman

Submission deadline: 24-Nov-2025 08:30AM

Information Security

BS Computer Science

BCS4A&B

Instructor: Sabghat Ullah Khan

Department of Computer Science

CUI Islamabad

Date: November 16, 2025

Contents

1	Question 1: Implement Classic Diffie–Hellman (DH)	4
1.1	Problem Statement	4
1.2	Restriction	4
1.3	Test Parameters	4
1.4	Expected Output	4
1.5	Deliverables	4
2	Question 2: DH Man-in-the-Middle Simulation	5
2.1	Problem Statement	5
2.2	Attack Diagram	5
2.3	Required Output	5
2.4	Analysis Question	5
2.5	Deliverables	5
3	Question 3: Implement ElGamal Over Integers	6
3.1	Problem Statement	6
3.2	Test Parameters	6
3.3	Message to Encrypt	6
3.4	Implementation Requirements	6
3.5	Deliverables	6
4	Question 4: ElGamal Attack – Reused k	7
4.1	Problem Statement	7
4.2	Mathematical Attack	7
4.2.1	Recovery Method	7
4.3	Hint	7
4.4	Expected Output	7
4.5	Deliverables	8
5	Question 5: Combined DH + ElGamal Hybrid Encryption	9
5.1	Problem Statement	9
5.2	Protocol Steps	9
5.2.1	Encryption (Alice’s Side)	9
5.2.2	Decryption (Bob’s Side)	9
5.3	Mathematical Notation	9
5.4	Test Message	9
5.5	Deliverables	10
6	Question 6: Combined DH + ECC System	11
6.1	Problem Statement	11
6.2	Protocol Flow	11
6.2.1	Alice’s Side	11
6.2.2	Bob’s Side	11
6.2.3	Both Parties	11
6.3	ECC Signature (Simplified ECDSA)	11
6.3.1	Signing	11
6.3.2	Verification	12

6.4	Hash Function	12
6.5	Deliverables	12
7	Question 7: Combined Attack – Break the Entire System	13
7.1	Problem Statement	13
7.2	Given Information	13
7.3	Multi-Stage Attack	13
7.3.1	Stage 1: Break Diffie–Hellman	13
7.3.2	Stage 2: Break ElGamal	13
7.3.3	Stage 3: Break ECC Signature	14
7.4	Implementation Requirements	14
7.5	Expected Output	14
7.6	Deliverables	15
8	Question 8: Code Audit – Find and Fix Security Bugs	16
8.1	Problem Statement	16
8.2	Types of Bugs to Look For	16
8.3	Example Bug and Fix	18
8.3.1	Buggy Code	18
8.3.2	Fixed Code	18
8.3.3	Justification	18
8.4	Submission Format	18
8.5	Deliverables	18

Assignment Overview

Assignment Scenario

The Military Communication System

A secure military communication system uses a **hybrid cryptographic protocol** that combines:

- **Diffie–Hellman** for key exchange
- **ElGamal** for encrypting short commands
- **ECC (Elliptic Curve Cryptography)** for signatures and additional encryption

However, analysts suspect misconfigurations and vulnerabilities. Your job is to **implement**, **test**, and **break** this system through a series of progressive challenges.

General Instructions

Important

- All code must be written in **Python 3.x**
- **Do NOT** use external cryptographic libraries except for basic math operations
- You must code **everything from scratch**
- Properly comment your code
- Include test cases for each function
- Submit a brief report explaining your findings

1 Question 1: Implement Classic Diffie–Hellman (DH)

1.1 Problem Statement

Write Python code that implements the classic Diffie–Hellman key exchange protocol with the following components:

1. Select a prime p and generator g
2. Perform key exchange between Alice and Bob
3. Compute the shared key on both sides
4. Verify that both keys match

1.2 Restriction

Important

Do NOT use `pow(a, b, p)` built-in function. You must implement your own **fast modular exponentiation** algorithm using the square-and-multiply method.

1.3 Test Parameters

- $p = 467$ (prime)
- $g = 2$ (generator)

1.4 Expected Output

Your program should print:

```
1 Alice's private key: [a]
2 Alice's public key: [A]
3 Bob's private key: [b]
4 Bob's public key: [B]
5 Alice's computed shared key: [K]
6 Bob's computed shared key: [K]
7 Keys match: True
```

1.5 Deliverables

1. Python file: `q1_diffie_hellman.py`
2. Function: `mod_exp(base, exponent, modulus)`
3. Function: `diffie_hellman(p, g)`
4. Test output screenshot

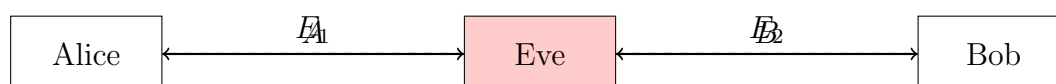
2 Question 2: DH Man-in-the-Middle Simulation

2.1 Problem Statement

Write Python code that simulates a **Man-in-the-Middle (MITM)** attack on the Diffie-Hellman protocol. The attack scenario is:

- Eve intercepts communication between Alice and Bob
- Eve generates two different fake keys:
 - $E_1 = g^{e_1} \bmod p$ sent to Alice
 - $E_2 = g^{e_2} \bmod p$ sent to Bob
- Alice and Bob believe they share a key, but Eve has **both** session keys

2.2 Attack Diagram



2.3 Required Output

Print the following:

```
1 --- Man-in-the-Middle Attack ---
2 Alice's computed key: [K_A]
3 Bob's computed key: [K_B]
4 Eve's key with Alice: [K_EA]
5 Eve's key with Bob: [K_EB]
6 Attack successful: Alice and Bob have different keys!
```

2.4 Analysis Question

In your report, explain:

1. Why is DH vulnerable to MITM without authentication?
2. What cryptographic mechanism can prevent this attack?
3. How would you modify DH to be secure?

2.5 Deliverables

1. Python file: q2_dh_mitm.py
2. Function: mitm_attack(p, g)
3. Written analysis (1 paragraph)

3 Question 3: Implement ElGamal Over Integers

3.1 Problem Statement

Implement a complete ElGamal encryption system with the following components:

1. **Key generation**
2. **Encryption**
3. **Decryption**
4. **Modular inverse** (Extended Euclidean Algorithm)
5. **Fast exponentiation**

3.2 Test Parameters

- $p = 31847$ (prime)
- $g = 5$ (generator)

3.3 Message to Encrypt

Encrypt the command:

"RETREAT AT 0900"

Encode each character by its ASCII value. Encrypt each character separately.

3.4 Implementation Requirements

Hint

Modular Inverse: Use the Extended Euclidean Algorithm to compute $a^{-1} \bmod p$. Do NOT use Python's `pow(a, -1, p)`.

3.5 Deliverables

1. Python file: `q3_elgamal.py`
2. Functions:
 - `elgamal_keygen(p, g)`
 - `elgamal_encrypt(m, p, g, h)`
 - `elgamal_decrypt(c1, c2, x, p)`
 - `mod_inverse(a, p)`
3. Output showing encryption/decryption of the message

4 Question 4: ElGamal Attack – Reused k

4.1 Problem Statement

Write Python code that:

1. Encrypts two different messages using the **same random value k**
2. Detects the reuse of k by comparing C_1 values
3. Recovers both plaintexts **without knowing the private key**

4.2 Mathematical Attack

When the same k is reused:

$$\text{Message 1: } (C_1, C_2) = (g^k, m_1 \cdot h^k) \quad (1)$$

$$\text{Message 2: } (C'_1, C'_2) = (g^k, m_2 \cdot h^k) \quad (2)$$

Notice: $C_1 = C'_1$ (same!)

4.2.1 Recovery Method

$$\frac{C_2}{C'_2} = \frac{m_1 \cdot h^k}{m_2 \cdot h^k} = \frac{m_1}{m_2} \quad (3)$$

$$C_2 = C'_2 \cdot \frac{m_1}{m_2} \quad (4)$$

$$m_1 = m_2 \cdot \frac{C_2}{C'_2} \bmod p \quad (5)$$

If you know one message, you can recover the other!

4.3 Hint

Hint

Try encrypting known messages like:

- Message 1: Known plaintext (e.g., "ATTACK")
- Message 2: Unknown plaintext

Use the ratio of ciphertexts to deduce the unknown message.

4.4 Expected Output

```
1 --- ElGamal Reused k Attack ---
2 Detected: C1 values are identical!
3 k was reused.
4
5 Message 1: [m1]
```



```
6 Message 2: [m2]
7
8 Ciphertext 1: (C1, C2)
9 Ciphertext 2: (C1, C2')
10
11 Ratio C2/C2': [ratio]
12 Recovered Message 2 (from known Message 1): [recovered_m2]
13 Attack successful!
```

4.5 Deliverables

1. Python file: q4_elgamal_attack.py
2. Function: attack_reused_k(c1_1, c2_1, c1_2, c2_2, known_m1, p)
3. Explanation of why k reuse is dangerous

5 Question 5: Combined DH + ElGamal Hybrid Encryption

5.1 Problem Statement

Design and implement a **hybrid cryptosystem** that combines Diffie–Hellman and ElGamal:

1. Alice and Bob first perform **Diffie–Hellman key exchange**
2. The DH shared key is used as a symmetric key to **XOR-mask** the plaintext
3. The masked plaintext is then encrypted using **ElGamal**
4. Bob decrypts using ElGamal, then XOR-unmasks with the DH key

5.2 Protocol Steps

5.2.1 Encryption (Alice’s Side)

1. Perform DH to get shared key K_{DH}
2. Compute: $m' = m \oplus K_{DH}$
3. Encrypt m' using ElGamal: (C_1, C_2)
4. Send: (C_1, C_2)

5.2.2 Decryption (Bob’s Side)

1. Perform DH to get shared key K_{DH}
2. Decrypt (C_1, C_2) using ElGamal: get m'
3. Compute: $m = m' \oplus K_{DH}$

5.3 Mathematical Notation

$$K_{DH} = g^{ab} \bmod p \quad (6)$$

$$m' = m \oplus K_{DH} \quad (7)$$

$$(C_1, C_2) = \text{ElGamal}_{\text{encrypt}}(m') \quad (8)$$

$$m' = \text{ElGamal}_{\text{decrypt}}(C_1, C_2) \quad (9)$$

$$m = m' \oplus K_{DH} \quad (10)$$

5.4 Test Message

Encrypt the message: "HYBRID CRYPTO SYSTEM"

5.5 Deliverables

1. Python file: `q5_hybrid.py`
2. Functions:
 - `hybrid_encrypt(message, p, g, dh_private, elgamal_public)`
 - `hybrid_decrypt(c1, c2, p, g, dh_private, elgamal_private)`
3. Output showing successful encryption and decryption
4. Brief explanation of security advantages

6 Question 6: Combined DH + ECC System

6.1 Problem Statement

Combine Diffie–Hellman and ECC to create a **secure authenticated key exchange**:

1. Perform DH to generate a symmetric key
2. Use ECC to **sign** the DH public keys (like ECDSA)
3. Verify ECC signatures before accepting DH keys
4. Only then compute the shared key

6.2 Protocol Flow

6.2.1 Alice's Side

1. Generate DH public key: $A = g^a \bmod p$
2. Sign A using ECC private key: σ_A
3. Send (A, σ_A) to Bob

6.2.2 Bob's Side

1. Generate DH public key: $B = g^b \bmod p$
2. Sign B using ECC private key: σ_B
3. Send (B, σ_B) to Alice

6.2.3 Both Parties

1. Verify the received signature
2. If valid, compute shared key: $K = g^{ab} \bmod p$
3. If invalid, reject the exchange

6.3 ECC Signature (Simplified ECDSA)

6.3.1 Signing

Choose random k (11)

$$(x_1, y_1) = kG \quad (12)$$

$$r = x_1 \bmod n \quad (13)$$

$$s = k^{-1}(H(m) + dr) \bmod n \quad (14)$$

Signature: (r, s)

6.3.2 Verification

$$w = s^{-1} \bmod n \quad (15)$$

$$u_1 = H(m) \cdot w \bmod n \quad (16)$$

$$u_2 = r \cdot w \bmod n \quad (17)$$

$$(x_1, y_1) = u_1 G + u_2 Q \quad (18)$$

$$\text{Verify: } r \stackrel{?}{=} x_1 \bmod n \quad (19)$$

6.4 Hash Function

Use a simple hash:

$$H(m) = m \bmod n \quad (20)$$

(In practice, use SHA-256)

6.5 Deliverables

1. Python file: `q8_dh_ecc.py`
2. Functions:
 - `ecc_sign(message, d, G, n, a, p)`
 - `ecc_verify(message, signature, Q, G, n, a, p)`
 - `authenticated_dh(p, g, curve_params)`
3. Output showing successful authenticated key exchange
4. Demonstrate rejection of invalid signatures

7 Question 7: Combined Attack – Break the Entire System

7.1 Problem Statement

A developer made **three critical mistakes**:

1. Reused the same ElGamal k value for multiple encryptions
2. Used a small/weak ECC curve
3. Performed unauthenticated Diffie–Hellman (vulnerable to MITM)

Your task: **Break all three components and recover all secrets.**

7.2 Given Information

You are provided with:

- DH public values: A and B
- Two ElGamal ciphertext pairs (both using the same k)
- ECC signatures that reuse a nonce (similar to the PlayStation 3 ECDSA vulnerability)

7.3 Multi-Stage Attack

7.3.1 Stage 1: Break Diffie–Hellman

Perform Man-in-the-Middle attack:

1. Intercept Alice’s public key A
2. Intercept Bob’s public key B
3. Compute both session keys

7.3.2 Stage 2: Break ElGamal

Exploit reused k :

1. Detect that C_1 values are identical
2. Use known plaintext attack
3. Recover the unknown plaintext

7.3.3 Stage 3: Break ECC Signature

Exploit nonce reuse (like Sony PS3 attack):

Given two signatures on different messages with the same k :

$$s_1 = k^{-1}(H(m_1) + dr) \bmod n \quad (21)$$

$$s_2 = k^{-1}(H(m_2) + dr) \bmod n \quad (22)$$

Recover k :

$$k = \frac{H(m_1) - H(m_2)}{s_1 - s_2} \bmod n \quad (23)$$

Then recover private key d :

$$d = \frac{s_1 k - H(m_1)}{r} \bmod n \quad (24)$$

7.4 Implementation Requirements

Write a single comprehensive script that:

1. Sets up the vulnerable system
2. Performs all three attacks sequentially
3. Prints all recovered secrets:
 - Both DH session keys
 - ElGamal plaintext
 - ECC private key

7.5 Expected Output

```

1 =====
2   COMBINED ATTACK - BREAKING ALL THREE
3   =====
4
5 [Stage 1: Breaking Diffie-Hellman]
6 Performing MITM attack...
7 Alice's session key: [K_A]
8 Bob's session key: [K_B]
9 Eve's keys: [K_EA], [K_EB]
10 DH broken!
11
12 [Stage 2: Breaking ElGamal]
13 Detecting reused k...
14 C1 values match: True
15 Recovering plaintext...
16 Known message 1: "ATTACK"
17 Recovered message 2: "DEFEND"
18 ElGamal broken!
19
20 [Stage 3: Breaking ECC Signatures]
21 Detecting nonce reuse...
22 Signature 1: (r1, s1)

```

```
23 Signature 2: (r2, s2)
24 r1 == r2: True (nonce reused!)
25 Recovering nonce k...
26 k = [recovered_k]
27 Recovering private key d...
28 d = [recovered_d]
29 Verifying: dG = Q ? True
30 ECC broken!
31
32 =====
33 ALL SECRETS RECOVERED!
34 System completely compromised.
35 =====
```

7.6 Deliverables

1. Python file: q10_combined_attack.py
2. Comprehensive output showing all three attacks
3. Report section explaining each vulnerability and fix

8 Question 8: Code Audit – Find and Fix Security Bugs

8.1 Problem Statement

You will be provided (by instructor) with a buggy Python file containing multiple security vulnerabilities. Your task is to:

1. **Identify** all security bugs
2. **Fix** each bug
3. **Justify** each fix with 1–3 sentences

8.2 Types of Bugs to Look For

The buggy code may contain:

- **Weak k generation** (predictable random values)
- **Wrong ECC inverse logic** (incorrect modular inverse)
- **Incorrect DH prime check** (composite number used as prime)
- **ElGamal using small p** (vulnerable to brute force)
- **No randomness in DH keys** (hardcoded private keys)
- **Curve with singularity** (invalid elliptic curve)
- **Integer overflow issues**
- **Missing input validation**

```
1 import random
2
3 def modinv(a, p):
4     return pow(a, p-1, p)
5
6 def dh_key_exchange(p, g):
7     a = 2
8     b = 2
9
10    A = (g ** a) % p
11    B = (g ** b) % p
12
13    keyA = pow(B, a)
14    keyB = pow(A, b)
15
16    return keyA, keyB
17
18 def elgamal_encrypt(M, p, g, h):
19
20    k = 3
21    C1 = pow(g, k, p)
```

```
22
23
24     s = (h * k) % p
25
26     C2 = (M * s) % p
27     return C1, C2
28
29
30 def elgamal_decrypt(C1, C2, x, p):
31     s = pow(C1, x+1, p)
32
33     s_inv = modinv(s, p)
34     M = (C2 * s_inv) % p
35     return M
36
37 def ec_add(P, Q, a, p):
38     if P == None: return Q
39     if Q == None: return P
40
41     x1, y1 = P
42     x2, y2 = Q
43
44     m = 3*x1*x1 + a * pow(2*y1, p-2, p)
45
46     x3 = (m*m - x1) % p
47     y3 = m*(x1 - x3) % p
48
49     return (x3, y3)
50
51
52 def ec_scalar_mult(k, P, a, p):
53     R = None
54     for i in range(k):
55         R = ec_add(R, P, a, p)
56     return R
57
58 def is_curve_valid(a, b, p):
59     disc = (4*a*a*a + 27*b) % p
60     return disc != 0
61
62 def main():
63     p = 97
64     g = 5
65
66     print("=== DH Key Exchange ===")
67     k1, k2 = dh_key_exchange(p, g)
68     print("Alice Key:", k1)
69     print("Bob Key:", k2)
70
71     print("\n=== ElGamal Demo ===")
72     pub_h = pow(g, 7, p)
73     C1, C2 = elgamal_encrypt(42, p, g, pub_h)
74     print("Ciphertext:", C1, C2)
75     M = elgamal_decrypt(C1, C2, 7, p)
76     print("Recovered Message:", M)
77
78     print("\n=== ECC Operations ===")
79     a = 2
```

```
80     b = 3
81     P = (3, 6)
82     print("Check if curve valid:", is_curve_valid(a, b, p))
83     print("Scalar Multiply 5P:", ec_scalar_mult(5, P, a, p))
84
85
86 if __name__ == "__main__":
87     main()
```

8.3 Example Bug and Fix

8.3.1 Buggy Code

```
1 def generate_k():
2     # Generate random k for ElGamal
3     return 42 # Always returns the same value!
```

8.3.2 Fixed Code

```
1 import secrets
2
3 def generate_k(p):
4     # Generate cryptographically secure random k
5     # Must be in range [2, p-2]
6     return secrets.randbelow(p - 3) + 2
```

8.3.3 Justification

The original code always returned 42, making the encryption deterministic and vulnerable to known-plaintext attacks. The fixed version uses `secrets.randbelow()` for cryptographically secure randomness in the valid range.

8.4 Submission Format

Create a document with:

1. **Bug #N:** Description of the bug
2. **Location:** Function name and line number
3. **Security Impact:** What attack this enables
4. **Fix:** Corrected code
5. **Justification:** Why this fix works (1–3 sentences)

8.5 Deliverables

1. Annotated buggy code (highlighting issues)
2. Fixed Python file: `q11_fixed.py`
3. Must find at least 5 bugs

Good Luck!

This assignment is challenging but rewarding.
Take your time, test thoroughly, and ask questions.

Remember: *Understanding is more important than perfection.*